ReLU Variants: Leaky ReLU, PReLU, ELU and SELU

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. The dying ReLU problem . . . 3
   - 3.1 Why a dead node stops learning . . . 3
   - 3.2 What makes z negative . . . 5
   - 3.3 Why a dead node stays dead . . . 5
   - 3.4 Dead nodes in practice . . . 5
4. Three ways to prevent dead nodes . . . 7
5. Linear variants . . . 8
   - 5.1 Leaky ReLU . . . 8
   - 5.2 Parametric ReLU (PReLU) . . . 9
6. Non-linear variants . . . 10
   - 6.1 ELU . . . 10
   - 6.2 SELU . . . 11
   - 6.3 GELU and SiLU: smooth versions of ReLU . . . 13
7. Summary . . . 16
8. Sources . . . 16
9. Key terms . . . 17

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Activation functions; ReLU.
- **Leads to:** Decoder-only GPT.
- **Compare with:** ReLU.

1. Overview

Key point: A ReLU node whose weighted sum stays negative outputs 0 with slope 0, so it never learns again: it is dead. Four variants give negative inputs a non-zero output and slope, which keeps every node alive.

ReLU (G-1668) is the default **activation function** (G-165) for **hidden layers** (G-890; see the activation functions). ReLU's biggest weakness is the **dying ReLU problem** (G-650). This Note covers:

- what a dead node is, why it happens and why it is permanent;
- three fixes, the third being a change of activation;
- two **linear variants** (G-1098; Leaky ReLU, Parametric ReLU), which change ReLU's negative side by a straight line;
- two **non-linear variants** (G-1336; ELU, SELU), which use an exponential curve there.
- two smooth versions of ReLU used in transformers (GELU, **SiLU** (G-2266)).

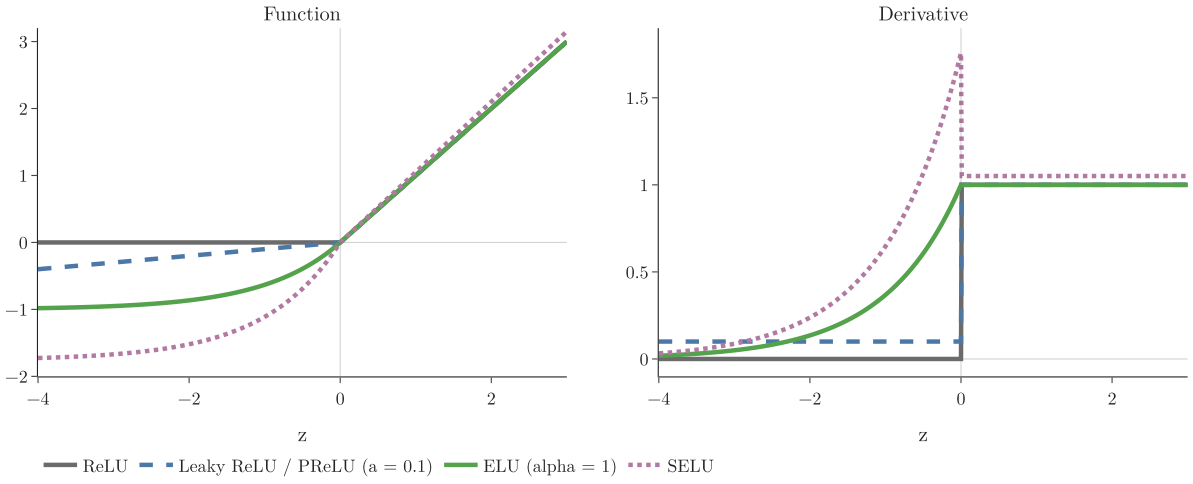


Figure 1: ReLU and its four variants (left) and their derivatives (right). On the negative side ReLU's slope is 0; every variant keeps a slope above 0. The Leaky ReLU slope is drawn as 0.1 instead of 0.01 to make it visible

Figure 1 shows all five functions. The left panel plots each function (output against the input z); the right panel plots each function's slope (its derivative, how steeply the left curve rises or falls at that z). The five agree for positive z (SELU is scaled up slightly) and differ only for negative z.

2. Prerequisites

- The activation functions: **ReLU** (G-1668), saturation, zero-centred outputs.
- The backpropagation how: gradients as chain-rule products.

3. The dying ReLU problem

> **Key point:** A dead node outputs 0 for every input. With more than half the nodes dead, the network cannot capture the patterns in the data; with all of them dead, there is no network left.

A **dead neuron** (G-551) is a node whose output is 0 for every input. The dead node's output no longer depends on the input, so it carries no information and learns nothing. Worse, it stays dead for the rest of training: in effect it has been removed from the network.

How much this matters depends on how many nodes die:

- **A few dead nodes:** the network has less **capacity** (G-344; the range of functions it can fit), but it still works.
- **More than half:** the network runs at less than half its size and cannot represent the patterns in the data well.
- **All of them:** the network outputs a constant. There is no point in training it.

3.1 Why a dead node stops learning

> **Key point:** If $z_1 < 0$, then $a_1 = 0$ and the slope $\partial a_1/\partial z_1$ is 0. The zero slope appears in the gradient of every weight into the node, so none of them is updated.

Take a small network for regression: two inputs, one hidden **ReLU** (G-1668) node, one output node (Figure 2). The hidden node computes

$$z_1 = w_1 x_1 + w_2 x_2 + b_1$$

$$a_1 = \max(0, z_1)$$

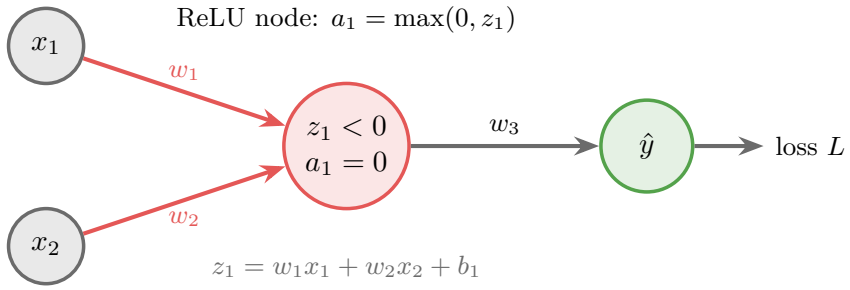


Figure 2: A ReLU node with $z_1 < 0$. The factor $\partial a_1/\partial z_1$ is 0, so the gradients of w_1 and w_2 are 0

Here L is the loss and $\hat{y}$ is the network's prediction. **Backpropagation** (G-247) reaches w_1 through the chain $L \to \hat{y} \to a_1 \to z_1 \to w_1$:

1. **In words:** the gradient of w_1 is a product of four factors, one of which is ReLU's slope.
2. **Formula:**
$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$
The gradient of w_2 has the same first three factors.
3. **Example:** with $w_1 = -0.8$, $w_2 = -0.5$, $b_1 = 0.1$ and inputs $x_1 = 0.6$, $x_2 = 0.4$:
$$w_1 x_1 = -0.8 \times 0.6 = -0.48$$
$$w_2 x_2 = -0.5 \times 0.4 = -0.20$$
$$z_1 = -0.48 - 0.20 + 0.1 = -0.58 < 0$$
$$\frac{\partial a_1}{\partial z_1} = 0$$
The last factor is the input $x_1 = 0.6$:
$$\frac{\partial z_1}{\partial w_1} = x_1 = 0.6$$
The product for w_1, factor by factor:
$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_1} \cdot 0 \cdot 0.6 = 0$$

So both gradients are 0:

$$\frac{\partial L}{\partial w_1} = 0$$

$$\frac{\partial L}{\partial w_2} = 0$$

The update changes nothing:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot 0$$

$$w_{\text{new}} = w_{\text{old}}$$

Neither weight changes. If z_1 is negative for every **observation** (G-1374; every record of the data), the node never gets an update again: it is dead.

3.2 What makes z negative

> **Key point:** Two causes: a learning rate so high that one update throws the weights negative, and a large negative bias.

1. A high learning rate (G-1068). Suppose that for the first observation z_1 is positive, so the gradient is not 0. With a very large learning rate, $\eta\,\partial L/\partial w$ is a big number. Subtracting it from a small weight makes w_1 and w_2 strongly negative, and in the next round z_1 is negative for every observation.

2. A large negative bias. If b_1 is very negative, $z_1 = w_1x_1 + w_2x_2 + b_1$ is negative however the inputs vary. The bias can start out negative, or it can be pushed there by updates, again usually because the learning rate is high.

3.3 Why a dead node stays dead

> **Key point:** The weights and bias no longer change, and the scaled inputs are too small to bring z back above 0.

Once z_1 is negative for every observation, nothing can make z_1 positive again:

- the weights and the bias get no updates, so they stay as they are;
- only the **features** (G-772) x_1, x_2 (the input variables) change from observation to observation, and they are scaled into a small range, so they cannot outweigh a large negative bias or negative weights.

For these two reasons a dead node is called permanently dead.

3.4 Dead nodes in practice

> **Key point:** On the moons data, a **learning rate** (G-1068) of 10 kills 41% and 69% of the nodes in two hidden layers, and a bias of -1 kills 72% and 100%; accuracy drops to guessing. **Leaky ReLU** (G-1064) and ELU with the same bias reach 89% and 96%.

The Notebook trains a network with two hidden layers of 32 nodes on 500 standardised (rescaled to mean 0 and standard deviation 1) observations of `make_moons` (G-111; two features, two classes), with plain **stochastic gradient descent (SGD)** (G-1892; here one update per mini-batch of 32 observations, so 16 updates per epoch) for 200 **epochs** (G-696; full passes over the data). The Notebook then counts the nodes whose z is negative for every training observation.

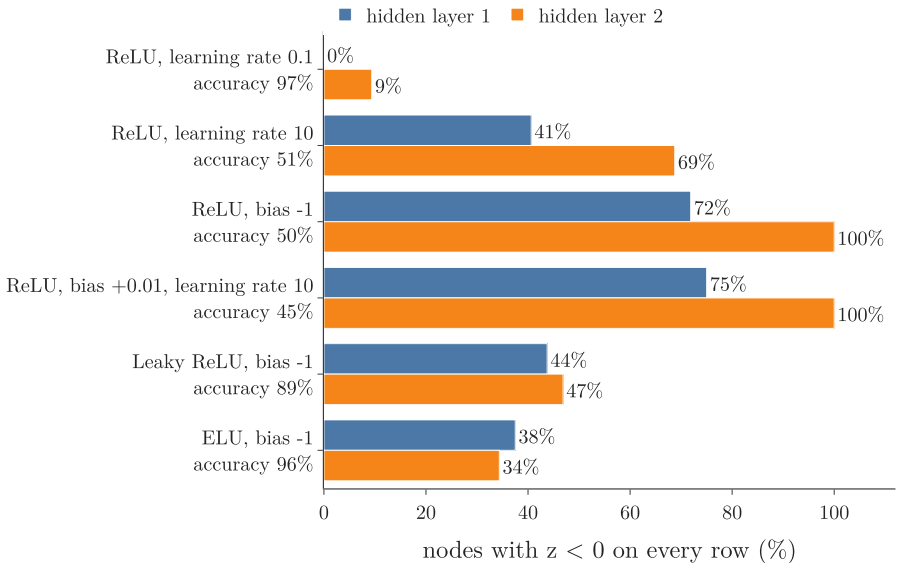


Figure 3: Share of nodes whose z is negative on every training observation, after 200 epochs, with the training accuracy. For ReLU these nodes are dead; for Leaky ReLU and ELU they still pass a gradient

Figure 3 shows the results:

- **ReLU, learning rate 0.1:** almost no dead nodes (0% and 9%), accuracy 97%.
- **ReLU, learning rate 10:** 41% and 69% dead, accuracy 51%. The nodes were already dead after the first epoch; the shares did not change in the 199 epochs after it.
- **ReLU, bias -1:** 72% and 100% dead from the very start, and still the same after 200 epochs. Accuracy 50%.

The deaths are permanent. Training the learning-rate-10 network for 20 more epochs changes the weights into its 13 dead first-layer nodes by exactly 0, as section 3.1 predicts.

Figure 4 follows the same runs through training, on a log scale (each step along the time axis multiplies the epoch by 10) so that the first epoch is visible.

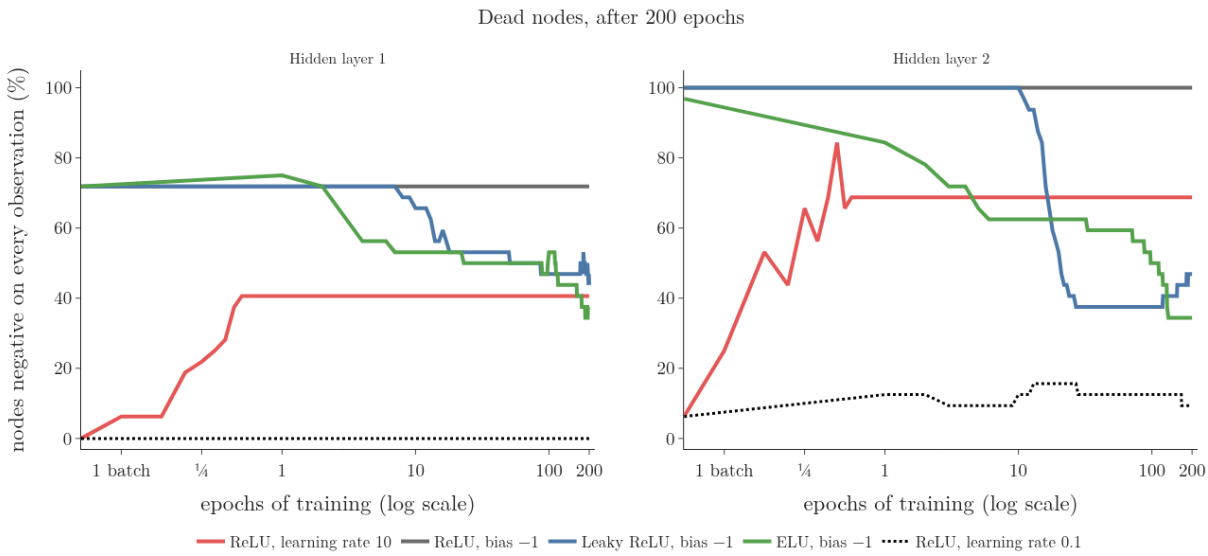


Figure 4: The share of nodes negative on every training observation during training, for the runs of Figure 3. The time axis is logarithmic; the learning-rate-10 run is also shown after each batch of its first epoch.

1. **Learning rate 10 (red).** The nodes die during the very first epoch. After 8 of its 16 batches, 41% of the first-layer nodes are negative everywhere; after that the share never moves again.
2. **Bias −1 with ReLU (grey).** 72% and 100% of the nodes start negative everywhere and stay so for all 200 epochs: a flat line.
3. **Bias −1 with Leaky ReLU (blue) and ELU (green).** These are two of the variants in Figure 1, defined in sections 5.1 and 6.1. The same nodes start negative, but their slope is not 0, so their weights keep changing. The ELU shares start falling within 3 epochs; the Leaky ReLU shares from epoch 8 in the first layer and epoch 11 in the second, where they drop from 100% to 38% by epoch 27.
4. **Learning rate 0.1 (dotted).** Few nodes are ever negative everywhere.

4. Three ways to prevent dead nodes

> **Key point:** Use a lower **learning rate** (G-1068), start the biases at a small positive value such as 0.01, or replace ReLU with one of its variants.

1. **A lower learning rate.** In Figure 3, changing only the learning rate, from 10 to 0.1, lowers the dead shares from 41% and 69% to 0% and 9%.
2. **A positive starting bias.** Starting every bias at a small positive value, such as 0.01 or 0.1, makes it very likely that every **ReLU** (G-1668) node starts active for most inputs (Goodfellow et al. 2016, §6.3.1, suggests 0.1).
3. **A ReLU variant.** The root cause is that ReLU's slope is exactly 0 for $z < 0$. The variants keep everything good about ReLU but give the negative side a slope.

Extra: A positive bias guards the start, not against a learning rate that is too high. Averaged over 5 seeds in the Notebook, a bias of +0.01 with learning rate 10 still ends with 49% and 80% dead nodes (bias 0: 49% and 89%), and accuracy stays near 50%. At learning rate 0.1, a starting bias of +0.1 lowers the share of second-layer nodes that start dead from 5.6% to 3.1%, and the share dead after training from 7.5% to 5.6%.

Python: Setting the starting bias in Keras.

```
keras.layers.Dense(
    32, activation="relu",
    bias_initializer=keras.initializers.Constant(0.01))
```

5. Linear variants

Key point: Leaky ReLU (G-1064) and Parametric ReLU replace the 0 on the negative side by a straight line az with a small slope a.

The **linear variants** of **ReLU** (G-1668) change only the negative side, and change it to a straight line.

5.1 Leaky ReLU

Key point: $f(z) = z$ for $z \geq 0$ and $0.01z$ for $z < 0$. Its slope on the negative side is 0.01 instead of 0, so a small gradient always flows.

1. **In words:** the **Leaky ReLU** (G-1064) keeps positive values and multiplies negative ones by 0.01 (Maas et al. 2013).
2. **Formula:**

$$f(z) = \begin{cases} z & z \geq 0 \\ 0.01\,z & z < 0 \end{cases}$$

$$f'(z) = \begin{cases} 1 & z \geq 0 \\ 0.01 & z < 0 \end{cases}$$

3. **Example:** $f(5) = 5$ and $f(-5) = -0.05$. At $z = -5$ **ReLU** (G-1668)'s slope would be 0; Leaky ReLU's is 0.01.

Because $\partial a/\partial z$ is never 0, the gradient in section 3.1 is never exactly 0. The weights keep changing a little, and a node can climb back out of the negative region: with a starting bias of -1, the share of first-layer nodes negative on every observation falls from 72% to 44% during training. In Figure 3, Leaky ReLU (Keras' `"leaky_relu"`, slope 0.2) with a starting bias of -1 reaches 89% accuracy, where ReLU stayed at 50%.

Advantages:

1. **Non-saturating** (G-1339) on both sides: the output is unbounded in both directions.
2. **Easy to compute:** no exponentials.
3. **No dying ReLU problem.**

4. **Close to zero-centred** (G-2148): outputs can be negative as well as positive, though not symmetric.

The only questionable point of Leaky ReLU is the constant: why 0.01 and not some other value? The value 0.01 was chosen by experiment. **Parametric ReLU** (G-1452) lets the data choose it instead.

> **Extra:** Keras does not use 0.01 by default (Keras documentation). The string `activation="leaky_relu"` uses slope 0.2, and the layer `keras.layers.LeakyReLU()` uses 0.3. To get 0.01, write `keras.layers.LeakyReLU(negative_slope=0.01)`.

5.2 Parametric ReLU (PReLU)

> **Key point:** Like **Leaky ReLU** (G-1064), but the negative slope a is a trainable parameter, learned per node along with the weights.

1. **In words:** the **Parametric ReLU (PReLU)** (G-1452) multiplies negative inputs by a slope a that is learned during training.
2. **Formula:**
$$f(z) = \begin{cases} z & z \geq 0 \\ a\,z & z < 0 \end{cases}$$
3. **Example:** if training sets $a = 0.25$, then $f(-2) = -0.5$; with $a = 0.01$ it would be Leaky ReLU, with $a = 0$ plain **ReLU** (G-1668).

The slope a is a parameter like a weight, found by gradient descent from the data. The slope a is not a **hyperparameter** (G-910) that we set. Everything else, advantages included, is as for Leaky ReLU. The extra flexibility can help: the paper that introduced PReLU reports better **ImageNet** (G-920; a large benchmark of labelled photos) accuracy than with ReLU (He et al. 2015).

> **Python:** PReLU is a separate layer after a `Dense` layer without activation.
> ```
> keras.layers.Dense(8),
> keras.layers.PReLU(),    # one learnable a per node
> ```
> Here the PReLU layer adds 8 trainable parameters, one slope per node. In Keras each starts at 0, so the network begins as a ReLU network (He et al. 2015 started them at 0.25).

> **Extra:** Nothing keeps the learned slope small. On the moons data the 8 slopes came out between -1.5 and 3.4. A negative a makes the function V-shaped, which is still a valid non-linear activation.

6. Non-linear variants

> **Key point:** ELU and SELU use an exponential curve for negative z that flattens out at a negative value. Outputs are **close to zero-centred** (G-2148) and there are no dead nodes.

The **non-linear variants** of **ReLU** (G-1668) use a curve, not a straight line, on the negative side.

6.1 ELU

> **Key point:** $f(z) = z$ for $z \geq 0$ and $\alpha(e^z - 1)$ for $z < 0$. Smooth, close to zero-centred, no dead nodes, often better test results than **ReLU** (G-1668); slower because of the exponential.

1. **In words:** the **ELU** (G-674; exponential linear unit; Clevert et al. 2016) is ReLU for positive z and an exponential curve for negative z that levels off at $-\alpha$.
2. **Formula:**

$$f(z) = \begin{cases} z & z \geq 0 \\ \alpha\left(e^z - 1\right) & z < 0 \end{cases}$$

$$f'(z) = \begin{cases} 1 & z \geq 0 \\ f(z) + \alpha & z < 0 \end{cases}$$

3. **Example:** with $\alpha = 1$ and $z = -1$:

$$f(-1) = e^{-1} - 1$$

$$f(-1) = 0.368 - 1 = -0.632$$

$$f'(-1) = f(-1) + 1 = -0.632 + 1 = 0.368$$

The negative-side slope comes from differentiating:

$$\frac{d}{dz}\,\alpha(e^z - 1) = \alpha e^z$$

$$\alpha e^z = f(z) + \alpha$$

A larger α pulls the negative side further down.

Advantages:

1. **Close to zero-centred**, so it converges faster.
2. **Better generalisation** (G-838): in experiments it often gives better results on test data than ReLU (Clevert et al. 2016).
3. **No dying ReLU problem:** the slope is above 0 for every z ($\alpha e^z > 0$ on the negative side). In Figure 3, ELU with a starting bias of -1 reaches 96%.
4. **Continuous and differentiable everywhere** (with $\alpha = 1$, which Figure 1 uses): there is no corner at 0.

Disadvantage: it needs an exponential, so it is slower to compute than ReLU. Faster convergence partly makes up for it, because fewer epochs are needed.

6.2 SELU

> **Key point:** SELU is ELU multiplied by a fixed $\lambda \approx 1.0507$, with fixed $\alpha \approx 1.6733$. Its outputs keep mean 0 and standard deviation 1 from layer to layer: it is self-normalising.

1. **In words:** the **SELU** (G-1766; scaled exponential linear unit; Klambauer et al. 2017) is ELU with a specific α, multiplied by a scale λ.
2. **Formula:**

$$f(z) = \lambda \begin{cases} z & z \geq 0 \\ \alpha\left(e^z - 1\right) & z < 0 \end{cases}$$

$$\lambda \approx 1.0507$$

$$\alpha \approx 1.6733$$

3. **Example:**

$$f(1) = 1.0507$$

$$\lambda\alpha = 1.0507 \times 1.6733 = 1.758$$

$$f(-1) = 1.758 \times (0.368 - 1) = -1.111$$

λ and α are fixed constants, not trainable parameters. The two constants were derived so that the function has one special property.

The special property is being **self-normalising** (G-1765): the outputs of a SELU layer have mean about 0 and standard deviation about 1, and the next layer keeps them there. Normalised values between layers make the network converge fast, and SELU also generalises well in experiments.

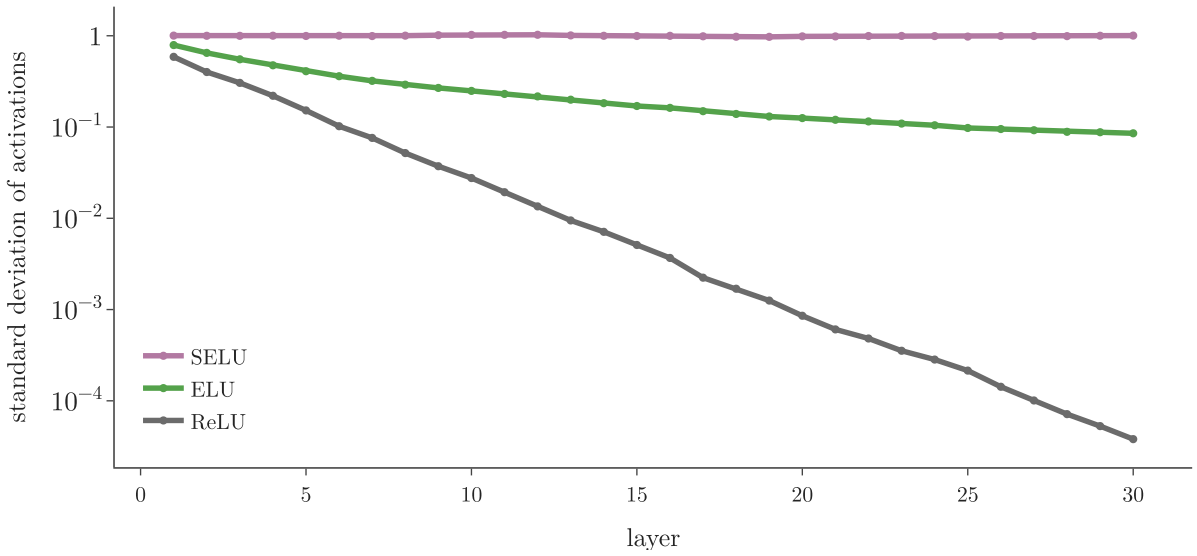


Figure 5: Standard deviation of the activations through 30 layers of 256 nodes, with no training. SELU stays at 1; ELU and ReLU shrink layer after layer

Figure 5 shows this in the Notebook: the horizontal axis is the layer number, and the vertical axis is the standard deviation of that layer's outputs (how spread out they are). Standard-normal inputs pass through 30 layers with the same random weights for each activation. With SELU the standard deviation is 1.00 at layer 1 and still 1.00 at layer 30. With **ReLU** (G-1668) it is 0.59 at layer 1 and 4×10^{-5} at layer 30.

The disadvantage of SELU is adoption. SELU is used in few places so far, for three reasons:

- SELU is recent (2017);
- its paper has 9 pages plus a 93-page appendix of proofs;
- less research builds on it.

Extra: Self-normalisation rests on assumptions in the paper: inputs with mean 0 and variance 1, and weights drawn with variance 1/inputs (Klambauer et al. 2017). Keras' documentation for `selu` therefore asks for `kernel_initializer="lecun_normal"` (as in Figure 5) and for `AlphaDropout` instead of ordinary **dropout** (G-639), which the paper shows disturbs the mean and variance. Starting weights are the subject of the weight initialisation.

Python: ELU and SELU in Keras.

```
keras.layers.Dense(32, activation="elu")
keras.layers.Dense(32, activation="selu",
                   kernel_initializer="lecun_normal")
```

6.3 GELU and SiLU: smooth versions of ReLU

> **Key point:** GELU is $z\,\Phi(z)$ and SiLU is $z\,\sigma(z)$: the input times a number between 0 and 1 that grows with the input. Both look like **ReLU** (G-1668) with a smooth bend and a small dip below 0. Transformers use them.

ReLU makes a hard choice: it keeps z when z is positive and replaces it by 0 when z is negative. A softer rule makes the same choice at random, with odds that depend on z:

1. **Keep or drop.** The node's value z is kept with probability $p(z)$ and replaced by 0 otherwise. This is dropout (G-639) with a rate that depends on the input.
2. **The odds follow the input.** A large positive z is almost always kept; a very negative z is almost always dropped.
3. **Take the average.** Over many random choices the average output is:

$$z \times p(z) + 0 \times (1 - p(z))$$

$$= z\,p(z)$$

Using this average as the **activation function** (G-165) needs no randomness at all.

The **GELU** (G-834; Gaussian error linear unit) uses for p the standard normal **cumulative distribution function** (G-515) $\Phi(z)$ (G-18): the probability that a standard normal value is below z. For example, $\Phi(0) = 0.50$ because half of the values are below 0, and $\Phi(1) = 0.84$ because 84 percent are below 1.

1. **In words:** the input times the probability of keeping it.
2. **Formula:**

$$\mathrm{GELU}(z) = z\,\Phi(z)$$

3. **Example:**

z	-2	-1	0	1	2
$\Phi(z)$, probability of being kept	0.02	0.16	0.50	0.84	0.98
GELU, $z\,\Phi(z)$	-0.05	-0.16	0	0.84	1.95

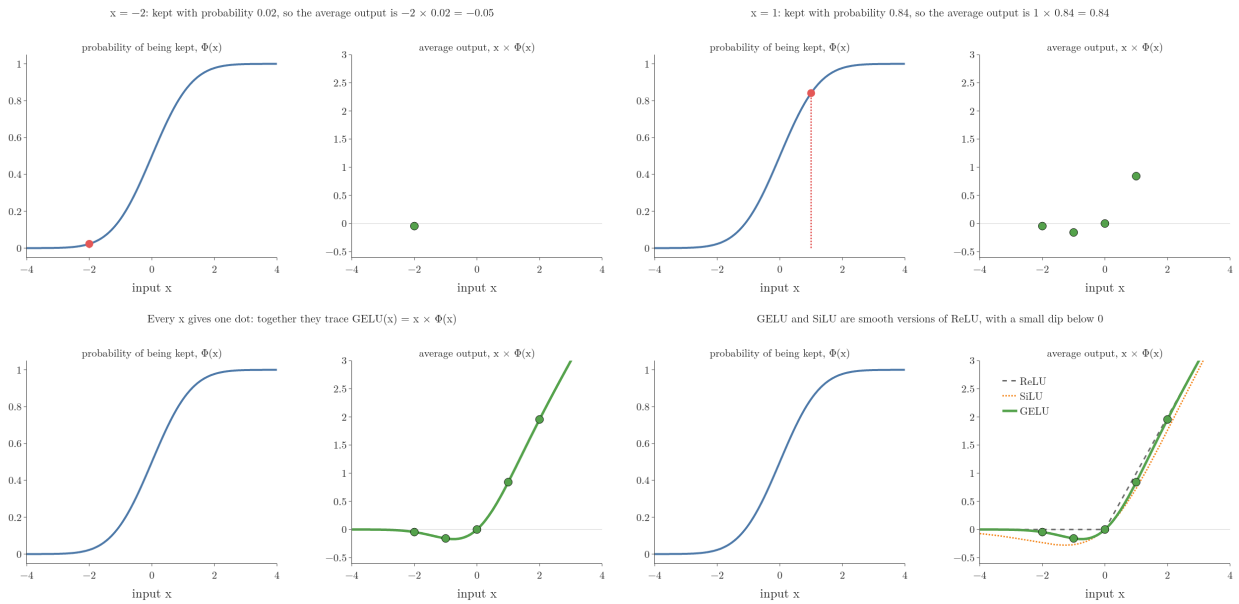


Figure 6: GELU built point by point. Left: the probability $\Phi(z)$ of keeping the input. Right: the average output $z\,\Phi(z)$, one dot per input, then the whole curve, then ReLU and SiLU for comparison. Idea after StatQuest, "The GELU, SiLU and SwiGLU activation functions, clearly explained!!!"

Figure 6 builds the curve from these five values. Watch the red dot on the left: its height is the number that multiplies the input on the right.

The **SiLU** (G-2266) (sigmoid linear unit, also called **Swish**) uses the **sigmoid** (G-1798; the S-shaped function that turns any number into 0 to 1) for p, a curve of almost the same shape as Φ:

$$\mathrm{SiLU}(z) = z\,\sigma(z)$$

For example:

$$\mathrm{SiLU}(-1) = -1 \times 0.269 = -0.27$$
$$\mathrm{SiLU}(1) = 1 \times 0.731 = 0.73$$

Compared with ReLU (last frame of Figure 6):

- **For large positive** z both are almost z, like ReLU, so they do not saturate there.
- **For negative** z the output is a small negative number, not 0: GELU dips to -0.17 and returns towards 0. The slope is not exactly 0, so a node with a slightly negative z still receives a gradient.
- **At 0** the curve bends smoothly; there is no corner.

GELU is the activation inside GPT's feed-forward layers; its use there is in section 6.4 of the **GPT** (G-855).

7. Summary

	ReLU	Leaky ReLU	PReLU	ELU	SELU
Negative side	0	$0.01z$	az, a learned	$\alpha(e^z - 1)$	$\lambda\alpha(e^z - 1)$
Kind		linear variant	linear variant	non-linear variant	non-linear variant
Slope for $z < 0$	0	0.01	a	$f(z) + \alpha$	$\lambda\alpha e^z$
Dead nodes	yes	no	no	no	no
Zero-centred	no	close	close	close	yes (self-normalising)
Exponential	no	no	no	yes	yes
Main drawback	dying	slope 0.01 is arbitrary		slower	not widely adopted

- A **ReLU** (G-1668) node with $z < 0$ for every observation has slope 0, gets no updates and is dead for good.
- Causes: a high **learning rate** (G-1068) and a large negative bias, because both can push z below 0 for every observation. Fixes: a lower learning rate, a bias starting at 0.01, or a variant.
- Linear variants (**Leaky ReLU** (G-1064), PReLU) put a straight line on the negative side; non-linear ones (ELU, SELU) a curve. Either way negative inputs keep a non-zero slope, so the node keeps learning.
- SELU keeps activations at mean 0 and standard deviation 1 across layers (**self-normalising** (G-1765)), so the network converges fast.
- GELU, $z\,\Phi(z)$, and **SiLU** (G-2266), $z\,\sigma(z)$, are smooth versions of ReLU: the input times a keep-probability that grows with the input.

8. Sources

Built from

- CampusX, "Relu Variants Explained | Leaky Relu | Parametric Relu | Elu | Selu | Activation Functions Part 2", YouTube, https://www.youtube.com/watch?v=2OwWs7Hzr9g
- StatQuest with Josh Starmer, "The GELU, SiLU and SwiGLU activation functions, clearly explained!!!", YouTube, https://www.youtube.com/watch?v=2FaI2Fen1mQ (section 6.3: GELU and SiLU as the average of input-dependent dropout, Figure 6)

Other references

- Clevert, D.-A., Unterthiner, T. and Hochreiter, S. (2016). Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR 2016. arXiv:1511.07289.
- He, K., Zhang, X., Ren, S. and Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV 2015. arXiv:1502.01852.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, section 6.3.1 (a small positive starting bias keeps ReLU units active at the start).

- Keras API documentation: `LeakyReLU`, `PReLU`, `selu` (keras.io/api).
- Klambauer, G., Unterthiner, T., Mayr, A. and Hochreiter, S. (2017). Self-Normalizing Neural Networks. NeurIPS 2017. arXiv:1706.02515.
- Maas, A. L., Hannun, A. Y. and Ng, A. Y. (2013). Rectifier Nonlinearities Improve Neural Network Acoustic Models. ICML 2013 Workshop on Deep Learning for Audio, Speech and Language Processing.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Dying ReLU problem (G-650)	ReLU nodes ending up with a negative weighted sum for every input, so they output 0 and stop learning.
Linear variants of ReLU (G-1098)	ReLU variants that replace ReLU's flat zero on the negative side with a straight line of small slope (Leaky ReLU and PReLU), so a node with negative inputs still has a gradient and keeps learning.
Non-linear variants of ReLU (G-1336)	ReLU variants with a curve on the negative side: ELU and SELU.
SiLU (Swish) (G-2266)	A smooth version of ReLU that multiplies each input by its sigmoid, $z\,\sigma(z)$; it is used as an activation and is close to GELU. SiLU stands for sigmoid linear unit; it is also called Swish.
Dead neuron (G-551)	A node whose output is 0 for every input; it gets no updates and stays that way.
Leaky ReLU (G-1064)	A ReLU variant with a small slope for negative inputs, so nodes do not die.
`make_moons` (G-111)	scikit-learn function that generates two interleaving half-moon classes.
Parametric ReLU (PReLU) (G-1452)	An activation that passes positive inputs unchanged and multiplies negative ones by a small slope a that is learned during training, one per node (a Leaky ReLU with a learned slope).

Term	Meaning
ELU (G-674)	Exponential linear unit: an activation that equals ReLU for positive inputs and follows a smooth exponential curve levelling off at $-\alpha$ for negative ones, so negative inputs still give a gradient and nodes do not die. Formula: z for $z \geq 0$, $\alpha(e^z - 1)$ for $z < 0$.
SELU (G-1766)	Scaled exponential linear unit: an activation that is ELU with $\alpha \approx 1.6733$, multiplied by $\lambda \approx 1.0507$; these constants make a network self-normalising, keeping each layer's outputs near mean 0 and standard deviation 1.
Self-normalising (G-1765)	Keeping the activations of every layer at mean 0 and standard deviation 1 without a separate normalisation step.
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
Activation function (G-165)	The function that turns a node's weighted sum z into its output, bringing it into a fixed range.
Hidden layer (G-890)	A layer of neurons between the input and output layers; it works on the output of the layer before and passes what it finds on, so the network can build up more complex patterns.
Capacity (G-344)	A model's ability to fit a wide variety of functions; too much capacity for the data leads to overfitting.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.
Observation (G-1374)	One record of a dataset: one row of the data table.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).

Term	Meaning
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.
Epoch (G-696)	One full update of the parameters using the whole training set.
Non-saturating function (G-1339)	A function with no ceiling on its output, so it does not flatten out, such as ReLU for positive inputs; its slope does not shrink to 0, which avoids the vanishing gradient that saturating functions cause.
Zero-centred activation (G-2148)	An activation whose outputs average about 0 over a layer, positive and negative; it feeds the next layer roughly centred inputs, which helps training converge faster.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
ImageNet (G-920)	A very large labelled image dataset with a yearly classification competition.
Generalisation (generalise) (G-838)	How well a model performs on new data it was not trained on.
Dropout (G-639)	Switching off a random set of input and hidden nodes at every training step, to reduce overfitting.
GELU (G-834)	A smooth activation function that multiplies each input x by $\Phi(x)$, the chance that a standard normal value is below x: $x\,\Phi(x)$ (Gaussian Error Linear Unit). GPT's MLPs use it.
Cumulative distribution function (CDF) (G-515)	For every value x, the probability that the random variable comes out at most x: $P(X \leq x)$.
$\Phi(z)$ (G-18)	The probability that a standard normal value is below z: the area under the bell curve to the left of z (the CDF of the standard normal distribution). For example $\Phi(1) = 0.84$.
Sigmoid function (G-1798)	An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1 + e^{-z})$; it turns a score into a probability, as in logistic regression.

Term	Meaning
GPT (G-855)	A language model that writes text by predicting the next token again and again (Generative Pre-trained Transformer); it is built from transformer decoder blocks only.